Weekly Mini Project 04 — Tasks & Team Workflow

1. Team

Member	Main Task	Area
Eyad Ibrahim (Team Lead)	Phase 0 Foundation + Prisma & Seed + common/ + Auth & Areas + Central MatchingService + PR Reviews & Integration	Backend & Foundation
Adham Albasha	Requests Module (sos, CRUD, cancel, eligible-donors) + Donations Module (prisma.$transaction) + Domain Logging	Backend
Laith Haj Hosin	Donors Module (/donors/me, eligible-requests, /donors) + Pledges Module (create, withdraw) + Endpoint Testing (api.http)	Backend
Molham Al Kasir	Shared UI & Utils + AuthContext + Sign In & Donor Sign Up + Public SOS Board (/) + Not Found (*)	Frontend
Taymaa Watfeh	DonorContext + Donor Dashboard (Eligibility Banner, Active Pledge, Eligible Requests, History) + Donor Profile & Deactivation	Frontend
Rami	RequestsContext + Coordinator Requests List & Filters + Create Request + Request Details (Update, Cancel, Record Donation) + Eligible Donors Screen	Frontend

Working pairs

Each frontend member works directly with their backend counterpart. All endpoints, request bodies, response shapes, and error messages are fixed in docs/API.md.

Feature Domain	Backend	Frontend
Foundation, Auth, Areas & Public SOS Board	Eyad (Auth & Areas), Adham (GET /requests/sos)	Molham

Feature Domain	Backend	Frontend
Donors, Eligibility & Pledges	Laith & Eyad (MatchingService)	Taymaa
Requests, Eligible Donors & Donations	Adham & Eyad (MatchingService)	Rami

Reviews

- **Eyad Ibrahim** reviews and approves the Pull Requests of all other team members.
- **Adham Albasha** reviews and approves Eyad's Pull Requests (GitHub does not allow self-approval).

2. Team Rules

Everyone must understand the whole project

Each member owns one area, but every member must be able to explain:

- the problem and product idea: matching urgent hospital blood requests with active, compatible, same-area donors outside their 56-day rest period (and why this is a coordination tool, not a medical fitness system)
- the 5 main user flows: Publish SOS request (Flow A) → Find eligible donors (Flow B) → Donor pledges (Flow C) → Record a donation in a transaction (Flow D) → Cancel a request (Flow E)
- the frontend routes, ProtectedRoute sign-in prompt, and the three contexts (AuthContext, DonorContext, RequestsContext)
- the backend architecture (Controller → Service → Persistence (Prisma)), DTO validation, JwtAuthGuard, and RolesGuard
- the red-cell compatibility table (R-1, R-2) and the 56-day rest period (REST_PERIOD_DAYS = 56, R-3..R-5)
- how eligible donors are filtered (R-6), ranked (R-7), and counted in the excluded summary (restPeriod, unavailable, inactive — R-8)
- all pledge rules (at most 1 active pledge per donor across all requests, active pledges cannot exceed unitsStillNeeded, automatic RELEASED on fulfillment or cancellation — R-15..R-18)
- why recording a donation uses prisma.$transaction (R-22) and what happens if any step fails
- how data persists in PostgreSQL through Prisma and what Swagger and AppLogger record

Every member speaks in the final English presentation (Section 18 of the specification).

Work alone, join through the contract

- Your work depends on docs/API.md and docs/PROJECT-STRUCTURE.md, not on someone's unfinished branch.
- In Phase 0, the team lead (Eyad) provides working **stubs**: every endpoint already answers with valid example JSON matching docs/API.md.
- Replace stubs only inside your own module.

Stay in your lane

- Edit only the files you own (section 4).
- Never push directly to main.
- Check your branch name before every commit (git branch --show-current).
- Never change a fixed name, route, DTO field, response shape, or error message without asking Eyad first.

3. Technology

Frontend

- React, TypeScript, Vite, React Router (react-router-dom)
- **React Context** for state (one context per feature: AuthContext, DonorContext, RequestsContext)
- Hooks: useState, useEffect, useCallback, useMemo, useContext
- Plain CSS, one file per page or component, using CSS variables from client/src/styles/theme.css and strict BEM owner prefixes
- API service layer in client/src/services/ (http.ts, auth.api.ts, donors.api.ts, pledges.api.ts, requests.api.ts)
- localStorage only for the JWT token (bdn.token inside http.ts)

Backend

- Node.js, NestJS, TypeScript
- **PostgreSQL** accessed through **Prisma ORM** (prisma/schema.prisma, migrations, prisma/seed.ts)
- Architecture: **Controller → Service → Persistence (Prisma)** (no business rules and no database queries in controllers)
- **JWT authentication** (@nestjs/jwt, bcrypt) with JwtAuthGuard and RolesGuard (COORDINATOR, DONOR, @Public())
- **DTO classes** with class-validator and class-transformer
- One global ValidationPipe (whitelist: true, forbidNonWhitelisted: true, stopAtFirstError: true) and one global AllExceptionsFilter
- **Swagger / OpenAPI** at /api/docs and custom AppLogger for domain events

Not allowed

- Redux, React Query, Zustand, or any external state library
- Tailwind CSS, Bootstrap, Material UI, or any external CSS/component library
- maps or external location services
- SMS, email, or push notifications
- payments, real hospital/lab systems, blood-unit inventory/expiry tracking, or medical fitness diagnosis
- WebSockets, GraphQL, Docker, or cloud deployment

- installing new npm packages without asking the team lead first

4. File Ownership

Every file has **one owner**. Do not edit another member's files.

Backend (api/)

Eyad Ibrahim (Team Lead)

```
api/prisma/
├── schema.prisma
├── migrations/
└── seed.ts

api/src/
├── main.ts
├── app.module.ts
├── prisma/
│   ├── prisma.module.ts
│   └── prisma.service.ts
├── common/
│   ├── constants.ts
│   ├── clock.service.ts
│   ├── app-logger.service.ts
│   ├── common.module.ts
│   ├── normalize.util.ts
│   ├── validators.ts
│   ├── validation.pipe.ts
│   └── all-exceptions.filter.ts
├── auth/
│   ├── decorators/ (public.decorator.ts, roles.decorator.ts, current-user.decorator.ts)
│   ├── guards/     (jwt-auth.guard.ts, roles.guard.ts)
│   ├── dto/        (sign-up.dto.ts, sign-in.dto.ts)
│   ├── auth.module.ts
│   ├── auth.controller.ts
│   ├── auth.service.ts
│   └── auth.types.ts
├── areas/
│   ├── areas.module.ts
│   ├── areas.controller.ts
│   └── areas.service.ts
└── matching/
    ├── matching.module.ts
    ├── matching.service.ts
    └── matching.types.ts
```

Adham Albasha

```
api/src/
├── requests/
│   ├── dto/
│   │   ├── create-request.dto.ts
│   │   ├── update-request.dto.ts
│   │   ├── cancel-request.dto.ts
│   │   ├── request-filters-query.dto.ts
│   │   └── request-id-param.dto.ts
│   ├── requests.module.ts
│   ├── requests.controller.ts
│   ├── requests.service.ts
│   └── requests.types.ts
└── donations/
    ├── dto/
    │   └── record-donation.dto.ts
    ├── donations.module.ts
    ├── donations.controller.ts
    └── donations.service.ts
```

Laith Haj Hosin

```
api/src/
├── donors/
│   ├── dto/
│   │   └── update-donor-profile.dto.ts
│   ├── donors.module.ts
│   ├── donors.controller.ts
│   ├── donors.service.ts
│   └── donors.types.ts
└── pledges/
    ├── dto/
    │   └── pledge-id-param.dto.ts
    ├── pledges.module.ts
    ├── pledges.controller.ts
    └── pledges.service.ts
```

api/requests/api.http

Frontend (client/)

Eyad Ibrahim (Team Lead)

```
client/src/
├── App.tsx
├── main.tsx
├── routes/AppRouter.tsx
├── styles/theme.css
├── styles/global.css
├── components/layout/
│   ├── Layout.tsx (+ css)
│   ├── Navbar.tsx (+ css)
│   └── ProtectedRoute.tsx (+ css)
├── components/common/PageContainer.tsx (+ css)
└── services/http.ts
```

Molham Al Kasir

```
client/src/
├── pages/SosBoard/
├── pages/SignIn/
├── pages/SignUp/
├── pages/NotFound/
├── context/AuthContext.tsx
├── services/auth.api.ts
├── types/auth.types.ts
├── components/common/
│   ├── Loading.tsx (+ css)
│   ├── ErrorMessage.tsx (+ css)
│   ├── EmptyState.tsx (+ css)
│   ├── Button.tsx (+ css)
│   ├── StatusBadge.tsx (+ css)
│   ├── UrgencyBadge.tsx (+ css)
│   └── BloodTypeBadge.tsx (+ css)
├── utils/format.ts
├── utils/date.ts
└── constants.ts
```

Taymaa Watfeh

```
client/src/
├── pages/DonorDashboard/
├── pages/DonorProfile/
├── context/DonorContext.tsx
├── services/donors.api.ts
├── services/pledges.api.ts
```

└── types/donor.types.ts

Rami

client/src/
├── pages/Requests/
├── pages/RequestDetails/
├── pages/EligibleDonors/
├── context/RequestsContext.tsx
├── services/requests.api.ts
└── types/request.types.ts

Repository and Docs

Eyad Ibrahim

docs/ README.md .github/ .editorconfig .prettierrc .gitignore api/package.json client/package.json

5. Phase 0 — Foundation (Eyad Ibrahim)

Eyad completes Phase 0 **before** the other five members start coding in feature branches. Target: **2 to 2.5 hours**. While Eyad sets up Phase 0, the other team members read the specification PDF and the three docs (PROJECT-STRUCTURE.md, API.md, TASKS.md), verify their local PostgreSQL setup, and prepare their environment.
The goal: every team member can branch from main, run npx prisma migrate dev && npx prisma db seed, start both api and client, and work in their own folder without touching shared files.

5.1 Repository

1. Create the GitHub repository (blood-donation-network) and invite all 5 team members.
2. Create the directory structure from docs/PROJECT-STRUCTURE.md.
3. Add .gitignore, .editorconfig, .prettierrc, and initial README.md.
4. Add docs/PROJECT-STRUCTURE.md, docs/API.md, and docs/TASKS.md.
5. Add .github/CODEOWNERS and .github/pull_request_template.md (section 15).
6. Make the initial foundation commit on main (one-time exception), push it, and **enable branch protection on main immediately** (section 15).

5.2 Database & Seed (api/prisma/)

1. Initialize Prisma in api/ with prisma/schema.prisma exactly as specified in PROJECT-STRUCTURE.md, section 4.

2. Run initial migration (npx prisma migrate dev --name init) and commit api/prisma/migrations/.
3. Write api/prisma/seed.ts creating:
    - 6 areas (Homs, Damascus, Aleppo, Latakia, Hama, Tartus)
    - 2 coordinator accounts (coordinator.homs@bloodnet.sy / 0911000001 and coordinator.damascus@bloodnet.sy / 0911000002, password Coordinator123!)
    - 24 donor accounts covering all 8 blood types (O-, O+, A-, A+, B-, B+, AB-, AB+), including **Sara** (A-, Homs, eligible) and **Rami** (O-, Homs, donated 20 days ago — in rest period), plus unavailable and inactive donors
    - 6 blood requests across OPEN, FULFILLED, and CANCELLED states (including the Homs A- CRITICAL 3-unit request)
4. Create @Global() PrismaModule and PrismaService in api/src/prisma/.

5.3 Backend (api/)

1. Create the NestJS project in api/ and install @prisma/client, prisma, class-validator, class-transformer, @nestjs/jwt, bcrypt, @types/bcrypt, and @nestjs/swagger.
2. Configure main.ts: global prefix api, CORS for http://localhost:5173, createValidationPipe(), AllExceptionsFilter, Swagger at /api/docs with .addBearerAuth(), port 3000.
3. Implement common/constants.ts, common/clock.service.ts, common/app-logger.service.ts, common/normalize.util.ts, common/validators.ts, common/validation.pipe.ts, common/all-exceptions.filter.ts, and common/common.module.ts.
4. Create module skeletons with the **fixed method names** (PROJECT-STRUCTURE.md, section 8.5): AreasModule, AuthModule, MatchingModule, RequestsModule, DonationsModule, DonorsModule, PledgesModule.
5. **Stubs**: Every method in RequestsService, DonationsService, DonorsService, and PledgesService returns valid example JSON from docs/API.md marked with // STUB: replace with real implementation.
6. Register all modules in app.module.ts and verify npm run start:dev starts cleanly and /api/docs loads.

5.4 Frontend (client/)

1. Create the Vite + React + TypeScript project in client/ and install react-router-dom.
2. Add styles/theme.css and styles/global.css (imported in main.tsx in that order).
3. Implement services/http.ts (request<T>(), ApiError, token helpers for localStorage key bdn.token) and create the four *.api.ts service files (auth.api.ts, donors.api.ts, pledges.api.ts, requests.api.ts) with all functions from API.md, section 18.
4. Create types/auth.types.ts, types/donor.types.ts, and types/request.types.ts exactly as in PROJECT-STRUCTURE.md, section 12.
5. Create AuthContext.tsx, DonorContext.tsx, and RequestsContext.tsx skeletons with the fixed value shapes so pages can import useAuth(), useDonor(), and useRequests() immediately.
6. Wrap AppRouter in App.tsx with AuthProvider, DonorProvider, and RequestsProvider.
7. Build components/layout/ (Layout, Navbar, ProtectedRoute) and

components/common/PageContainer.tsx, plus working placeholder components in components/common/ with the fixed props.
8. Configure routes/AppRouter.tsx with all 9 routes pointing to placeholder pages.
9. Add client/.env.example with VITE_API_URL=http://localhost:3000/api.

5.5 Hand-off

Phase 0 is complete when:

- main has the full backend and frontend skeleton
- npx prisma migrate dev and npx prisma db seed succeed on a clean database
- api (http://localhost:3000/api and /api/docs) and client (http://localhost:5173) both start cleanly
- every endpoint answers with stub data or foundation data
- every route renders its placeholder page
- branch protection is active on main
- Eyad announces in the team chat: **"Phase 0 is merged and protected — branch from main now."**

6. Task 1 — Eyad Ibrahim

Team Lead & Core Backend Architect: Prisma & Seed + common/ + Auth & Areas + Central MatchingService + Integration

Main responsibility

After completing Phase 0, finalize and unit-test the shared backend infrastructure (common/), the Authentication and Areas modules (auth/, areas/), and the **Central Matching & Eligibility Engine (MatchingService)** that both Adham (RequestsModule, DonationsModule) and Laith (DonorsModule, PledgesModule) call. Also review all Pull Requests and lead integration, README, and presentation preparation.

Files

See Eyad's backend, frontend layout, and repository files in section 4.

Part A — Shared Validation, Normalization, Exception Filter & Logger (api/src/common/)

Deliver and merge these first so Adham and Laith have all shared decorators and helpers ready:

1. **constants.ts**: REST_PERIOD_DAYS = 56, MIN_UNITS = 1, MAX_UNITS = 10, BLOOD_COMPATIBILITY_MAP, URGENCY_SORT_ORDER.

2. **validators.ts**:
    - @IsValidBloodType(): accepts only 'O-' | 'O+' | 'A-' | 'A+' | 'B-' | 'B+' | 'AB-' | 'AB+'.
    - @IsValidPhone(): accepts optional + followed by 8–15 digits after removing spaces and dashes.
    - @IsCalendarDate(): accepts YYYY-MM-DD and verifies that the year, month, and day form a real calendar date ('2026-02-31' returns false).
3. **normalize.util.ts**:
    - normalizePhone(value): strips spaces and dashes.
    - normalizeEmail(value): trims and lowercases.
    - toApiBloodType(prismaEnum) & toPrismaBloodType(apiBloodType): converts between A_NEG and 'A-'.
    - toDateOnlyString(date): formats a Date as YYYY-MM-DD in local time without UTC day shifting.
4. **validation.pipe.ts & all-exceptions.filter.ts**:
    - Enforces whitelist: true, forbidNonWhitelisted: true, stopAtFirstError: true.
    - Formats unknown fields as 400 { "message": "Unknown field '<property>' is not allowed." }.
    - Formats all HTTP exceptions, unknown routes (404 Endpoint not found.), invalid JSON (400 Invalid request body.), and unexpected errors (500 Something went wrong. Please try again.) into { "message": "..." }.
5. **app-logger.service.ts**:
    - Implements logRequestCreated, logRequestCancelled, logRequestFulfilled, logDonationRecorded, and warnDonationRejected. Never logs passwords or JWTs.

Part B — Authentication, Guards & Areas (api/src/auth/ & api/src/areas/)

1. **GET /api/areas (@Public())**: returns all areas ordered by name ASC.
2. **POST /api/auth/sign-up (@Public())**:
    - Validates SignUpDto in exact order (fullName, phone, email, password, bloodType, areaId).
    - Checks AreasService.findByIdOrThrow(dto.areaId) (400 Selected area is not supported.).
    - Checks duplicate phone (409 An account with this phone number is already registered.) and duplicate email (409 An account with this email address is already registered.).
    - Hashes password with bcrypt.hash(dto.password, 10).
    - Creates User (role: Role.DONOR) and DonorProfile in a transaction and returns { accessToken, user }.
3. **POST /api/auth/sign-in (@Public())**:
    - Looks up user by normalized phone or lowercased email.
    - Compares password with bcrypt.compare. If user not found or password mismatch, throws 401 Invalid credentials. Please check your phone/email and password. (never revealing which part was wrong).
    - If !user.isActive, throws 403 This account has been deactivated..
    - Signs JWT { sub: user.id, role: user.role } and returns { accessToken, user }.
4. **GET /api/auth/me**: returns the authenticated AuthUser.

5. **JwtAuthGuard & RolesGuard**:
    - JwtAuthGuard allows @Public() endpoints; otherwise verifies the Bearer token and checks user.isActive === true (401 Authentication required. Please sign in.).
    - RolesGuard checks @Roles(...) against req.user.role (403 You do not have permission to perform this action.).

Part C — Central Matching & Eligibility Engine (api/src/matching/matching.service.ts)

This is the core algorithmic service of the project.

1. Red-Cell Compatibility (R-1, R-2)

Blood type needed (request.bloodType)	Compatible donor blood types (BLOOD_COMPATIBILITY_MAP)
O-	O-
O+	O+, O-
A-	A-, O-
A+	A+, A-, O+, O-
B-	B-, O-
B+	B+, B-, O+, O-
AB-	AB-, A-, B-, O-
AB+	AB+, AB-, A+, A-, B+, B-, O+, O-

2. Rest Period (R-3, R-4, R-5)

- getNextAllowedDonationDate(lastDonationDate):
    - If lastDonationDate === null, returns null (R-5: a donor with no recorded donation has no rest period).
    - Otherwise adds REST_PERIOD_DAYS (56 days from common/constants.ts) to lastDonationDate and returns YYYY-MM-DD (R-3, R-4).
- isInRestPeriod(lastDonationDate, now):
    - Returns false if lastDonationDate === null.
    - Returns true while toDateOnlyString(now) < getNextAllowedDonationDate(lastDonationDate) (R-4). On or after nextAllowedDonationDate, returns false.

3. Single-Donor Eligibility Check (R-6, R-15, R-19)

checkDonorEligibilityForRequest(donor, request, now) checks these 5 conditions in order and returns both a forCoordinator message (used by DonationsService) and a forDonor message (used by PledgesService):

Check Order	Condition	Coordinator Message (POST /donations)	Donor Message (POST /pledges)
1	!donor.user.isActive	This donor account is deactivated and cannot donate.	This account has been deactivated.

Check Order	Condition	Coordinator Message (POST /donations)	Donor Message (POST /pledges)
2	isInRestPeriod(donor.lastDonationDate, now)	This donor is in rest period until YYYY-MM-DD.	You cannot pledge while in rest period. Your next allowed donation date is YYYY-MM-DD.
3	!donor.isAvailable	This donor is currently marked as unavailable.	You cannot pledge while your availability is set to unavailable.
4	!isBloodCompatible(request.bloodType, donor.bloodType)	This donor's blood type (<donorType>) is not compatible with <neededType>.	Your blood type (<donorType>) is not compatible with this request (<neededType>).
5	donor.areaId !== request.areaId	This donor is in <donorArea>, but the request is in <requestArea>.	You can only pledge to requests in your registered area (<donorArea>).

4. Ranked Eligible Donors & Excluded Summary (R-7, R-8)

getEligibleDonorsForRequest(requestId):

1. Queries all donors in request.areaId whose bloodType is in getCompatibleDonorTypes(request.bloodType).
2. Classifies each compatible same-area donor:
    - !donor.user.isActive → increments excludedSummary.inactive
    - isInRestPeriod(donor.lastDonationDate, now) → increments excludedSummary.restPeriod
    - !donor.isAvailable → increments excludedSummary.unavailable
    - otherwise → adds to eligibleDonors
3. Sorts eligibleDonors by:
    - Exact blood-type match (donor.bloodType === request.bloodType) first (true before false)
    - Oldest lastDonationDate first (null / never donated comes first; then earlier dates before later dates)
    - fullName alphabetically (a.fullName.localeCompare(b.fullName))

Test Table for MatchingService

Scenario	Input	Expected Output
Request needs A- in Homs	Donors: Sara (A-, never donated), Omar (O-, donated 120 days ago), Zaid (A-, donated 90 days ago), Rami (O-, donated 20 days ago), Lina (A+, never donated)	Eligible order: **1. Sara** (A-, never), **2. Zaid** (A-, 90d ago), **3. Omar** (O-, 120d ago). Rami counted in excludedSummary.restPeriod = 1. Lina (A+) ignored (incompatible).

Scenario	Input	Expected Output
Donor donated on 2026-11-01	lastDonationDate = 2026-11-01, REST_PERIOD_DAYS = 56	nextAllowedDonationDate = '2026-12-27'. On 2026-12-26, isInRestPeriod = true. On 2026-12-27, isInRestPeriod = false.
Two O- donors who never donated	"basel" and "Ahmad"	"Ahmad" ranks before "basel" (alphabetical tie-breaker).

Eyad must NOT

- write the controller/service implementation of RequestsModule, DonationsModule, DonorsModule, or PledgesModule (beyond Phase 0 stubs)
- build the feature pages owned by Molham, Taymaa, or Rami
- log passwords or JWT tokens in AppLogger
- allow 56 to be hardcoded outside common/constants.ts
- merge any PR that fails to build or touches unowned files

Definition of Done (Eyad)

- Phase 0 foundation is merged and branch protection is active on main.
- prisma/schema.prisma, migrations, and prisma/seed.ts (6 areas, 2 coordinators, 24 donors across all 8 types, 6 requests) run cleanly.
- ValidationPipe, AllExceptionsFilter, custom validators, ClockService, and AppLogger work as specified.
- POST /api/auth/sign-up, POST /api/auth/sign-in, GET /api/auth/me, GET /api/areas, JwtAuthGuard, and RolesGuard enforce all auth and role rules.
- MatchingService implements R-1..R-8 and passes every case in the test table.
- Layout, Navbar, and ProtectedRoute (showing the sign-in prompt when unauthenticated) work on desktop and mobile.
- All team PRs are reviewed promptly; README.md and the presentation rehearsal are complete.

7. Task 2 — Adham Albasha

Backend Requests & Donations (api/src/requests/ & api/src/donations/)

Main responsibility

Own the full blood request lifecycle (GET /requests/sos, POST /requests, GET /requests, GET /requests/:id, PATCH /requests/:id, PATCH /requests/:id/cancel, GET /requests/:id/eligible-donors) and the atomic donation recording endpoint (POST /requests/:id/donations) using prisma.$transaction.

Files

```
api/src/requests/
├── dto/
│   ├── create-request.dto.ts
│   ├── update-request.dto.ts
│   ├── cancel-request.dto.ts
│   ├── request-filters-query.dto.ts
│   └── request-id-param.dto.ts
├── requests.module.ts
├── requests.controller.ts
├── requests.service.ts
└── requests.types.ts

api/src/donations/
├── dto/
│   └── record-donation.dto.ts
├── donations.module.ts
├── donations.controller.ts
└── donations.service.ts
```

Part A — DTOs (First PR)

Declare properties in the exact order from PROJECT-STRUCTURE.md, section 8.4, with the exact error messages from docs/API.md:

- **CreateRequestDto**: patientReference, bloodType (@IsValidBloodType()), unitsRequired (@IsInt(), @Min(MIN_UNITS), @Max(MAX_UNITS)), areaId, urgency (@IsIn(['CRITICAL', 'HIGH', 'NORMAL'])), hospital, note (@IsOptional(), @IsString()).
- **UpdateRequestDto**: unitsRequired (optional, integer 1–10), urgency (optional), note (optional).
- **CancelRequestDto**: cancelReason (@Transform trim, @IsString(), @IsNotEmpty() → 'Cancellation reason is required.').
- **RequestFiltersQueryDto**: optional status, bloodType, areaId, urgency.
- **RecordDonationDto**: donorId ('Donor ID is required.'), donationDate (@IsCalendarDate() → 'Invalid date. Use the format YYYY-MM-DD.').

Part B — Requests Service & Controller (api/src/requests/)

1. **getPublicSosRequests() (GET /api/requests/sos, @Public())**:
    - Queries status: RequestStatus.OPEN.
    - Orders by urgency (CRITICAL → HIGH → NORMAL) and then createdAt ASC (oldest first, FR-8).
    - Maps to PublicSosRequest (id, bloodType, area, urgency, hospital, unitsStillNeeded, createdAt). **Never** includes patientReference, note, pledges, or

donations (R-23).

2. **createRequest(coordinatorId, dto) (POST /api/requests, @Roles('COORDINATOR'))**:
    - Verifies areaId exists via AreasService.findByIdOrThrow(dto.areaId).
    - Checks for an existing OPEN request with the same patientReference (case-insensitive) and bloodType (R-13). If found, throws ConflictException('An open request for this patient reference and blood type already exists.').
    - Creates the OPEN request with unitsCollected: 0, calls AppLogger.logRequestCreated(...), and returns CoordinatorRequestDetails.
3. **findAllRequests(filters) (GET /api/requests, @Roles('COORDINATOR'))**:
    - Applies optional status, bloodType, areaId, and urgency filters (FR-7) and returns CoordinatorRequestSummary[] with calculated unitsStillNeeded and activePledgesCount.
4. **findRequestById(id) (GET /api/requests/:id, @Roles('COORDINATOR'))**:
    - Returns CoordinatorRequestDetails including pledges and donations (FR-13), or throws NotFoundException('Blood request not found.').
5. **updateRequest(id, dto) (PATCH /api/requests/:id, @Roles('COORDINATOR'))**:
    - Verifies request exists (404) and is OPEN (409 Only OPEN requests can be updated. — R-9, R-14).
    - If dto.unitsRequired !== undefined && dto.unitsRequired < request.unitsCollected, throws BadRequestException('Units required cannot be reduced below the units already collected.') (R-12).
    - Inside prisma.$transaction: updates the request; if the updated unitsRequired === request.unitsCollected and request.unitsCollected > 0, sets status = RequestStatus.FULFILLED, updates all PLEDGED pledges on this request to RELEASED (R-11, R-18), and logs AppLogger.logRequestFulfilled(id).
6. **cancelRequest(id, dto) (PATCH /api/requests/:id/cancel, @Roles('COORDINATOR'))**:
    - Verifies request exists (404) and is OPEN (409 Only OPEN requests can be cancelled. — R-9, R-14).
    - Inside prisma.$transaction: sets status = RequestStatus.CANCELLED, saves cancelReason, and updates all PLEDGED pledges on this request to RELEASED (Flow E, R-18). Calls AppLogger.logRequestCancelled(id, dto.cancelReason).
7. **getEligibleDonors(id) (GET /api/requests/:id/eligible-donors, @Roles('COORDINATOR'))**:
    - Verifies request exists (404) and is OPEN (409 Eligible donors can only be matched for OPEN requests. — R-9).
    - Delegates to MatchingService.getEligibleDonorsForRequest(id) (Flow B, FR-9).

Part C — Atomic Donation Recording (api/src/donations/)

recordDonation(requestId, coordinatorId, dto) (POST /api/requests/:id/donations, @Roles('COORDINATOR')):

1. Load request (404 Blood request not found.).
2. Check request.status === RequestStatus.OPEN. If not, call AppLogger.warnDonationRejected(requestId, dto.donorId, 'Donations can only be recorded for OPEN requests.') and throw 409 Donations can only be recorded for OPEN requests. (R-9, R-19).

3. Check dto.donationDate <= toDateOnlyString(ClockService.now()). If in the future, call AppLogger.warnDonationRejected(requestId, dto.donorId, 'The donation date cannot be in the future.') and throw 400 The donation date cannot be in the future. (R-21).
4. Load donor with user and area (404 Donor not found.).
5. Call MatchingService.checkDonorEligibilityForRequest(donor, request, ClockService.now()) (R-19). If !check.eligible, call AppLogger.warnDonationRejected(requestId, dto.donorId, check.forCoordinator) and throw BadRequestException(check.forCoordinator).
6. Execute inside await this.prisma.$transaction(async (tx) => { ... }) (R-22):
    - Create Donation (donorId, requestId, recordedById: coordinatorId, donationDate: new Date(dto.donationDate), units: 1) (R-20).
    - Increment unitsCollected: const newUnitsCollected = request.unitsCollected + 1.
    - Complete the donor's active pledge on this request (if any): tx.pledge.updateMany({ where: { requestId, donorId: donor.id, status: PledgeStatus.PLEDGED }, data: { status: PledgeStatus.COMPLETED } }).
    - Update donor's lastDonationDate to new Date(dto.donationDate).
    - If newUnitsCollected >= request.unitsRequired:
        - Update request status to RequestStatus.FULFILLED (R-11).
        - Release remaining pledges: tx.pledge.updateMany({ where: { requestId, status: PledgeStatus.PLEDGED }, data: { status: PledgeStatus.RELEASED } }) (R-18).
    - Otherwise, update request unitsCollected: newUnitsCollected.
7. Call AppLogger.logDonationRecorded(...) (and logRequestFulfilled(requestId) if fulfilled) and return the updated CoordinatorRequestDetails.

Test Table (Adham)

Case	Expected Result
GET /api/requests/sos	Only OPEN requests, ordered CRITICAL → HIGH → NORMAL then createdAt ASC, zero patientReference or note fields
Second OPEN request with same patientReference + bloodType	409 An open request for this patient reference and blood type already exists.
Second OPEN request with same patientReference but different bloodType (or after first is FULFILLED/CANCELLED)	201 Created (allowed by R-13)
PATCH /api/requests/:id with unitsRequired: 1 when unitsCollected: 2	400 Units required cannot be reduced below the units already collected.
PATCH /api/requests/:id with unitsRequired: 2 when unitsCollected: 2	200 OK, status becomes FULFILLED, remaining PLEDGED pledges become RELEASED
POST /api/requests/:id/donations for 1st of 3 units (Sara pledged)	201 Created, unitsCollected = 1, unitsStillNeeded = 2, Sara's pledge COMPLETED, Sara's lastDonationDate updated
POST /api/requests/:id/donations for Sara again right away	400 This donor is in rest period until YYYY-MM-DD., logged as WARN, nothing

Case	Expected Result
	saved in DB
POST /api/requests/:id/donations for 3rd of 3 units (when another donor holds a PLEDGED pledge)	201 Created, unitsCollected = 3, unitsStillNeeded = 0, request FULFILLED, other donor's pledge RELEASED

Adham must NOT

- duplicate blood compatibility or 56-day rest period math inside RequestsService or DonationsService (always call Eyad's MatchingService)
- perform multi-step writes in recordDonation, updateRequest, or cancelRequest outside prisma.$transaction
- expose patientReference, note, or donor data on GET /api/requests/sos
- put business logic or PrismaService calls inside controllers
- write frontend code or edit files outside api/src/requests/ and api/src/donations/

Definition of Done (Adham)

- All 5 DTOs validate format and property order with exact API.md messages and Swagger @ApiProperty decorators.
- GET /api/requests/sos is public, ordered properly, and leaks zero personal/internal data.
- POST /api/requests rejects duplicate OPEN requests (patientReference + bloodType) with 409.
- GET /api/requests supports all 4 filters (status, bloodType, areaId, urgency).
- PATCH /api/requests/:id and PATCH /api/requests/:id/cancel enforce OPEN-only rules, unitsRequired >= unitsCollected, auto-fulfillment, and pledge release.
- POST /api/requests/:id/donations runs atomically inside prisma.$transaction and logs recorded and rejected donations via AppLogger.
- Every endpoint is documented in Swagger with @ApiTags, @ApiOperation, and @ApiBearerAuth.

8. Task 3 — Laith Haj Hosin

Backend Donors, Pledges & Endpoint Testing (api/src/donors/ + api/src/pledges/ + api/requests/api.http)

Main responsibility

Own the Donor endpoints (/donors/me, /donors/me/deactivate, /donors/me/eligible-requests, /donors), the Pledge endpoints (POST /requests/:id/pledges, PATCH /pledges/:id/withdraw), and the comprehensive REST Client test suite (api/requests/api.http).

Files

```
api/src/donors/
├── dto/
│   └── update-donor-profile.dto.ts
├── donors.module.ts
├── donors.controller.ts
├── donors.service.ts
└── donors.types.ts

api/src/pledges/
├── dto/
│   └── pledge-id-param.dto.ts
├── pledges.module.ts
├── pledges.controller.ts
└── pledges.service.ts

api/requests/api.http
```

Part A — Donors Service & Controller (api/src/donors/)

1. **getMyProfile(userId) (GET /api/donors/me, @Roles('DONOR'))**:
   - Loads the donor profile for userId along with user, area, active PLEDGED pledge (including its request and request.area), and donations (ordered by donationDate DESC, including request and request.area).
   - Uses MatchingService.getNextAllowedDonationDate(donor.lastDonationDate) and MatchingService.getDonorEligibilityStatus(donor, ClockService.now()) to compute nextAllowedDonationDate and eligibilityStatus (ELIGIBLE / REST_PERIOD / UNAVAILABLE / INACTIVE) (FR-3, FR-4, R-24).
2. **updateMyProfile(userId, dto) (PATCH /api/donors/me, @Roles('DONOR'))**:
   - Validates UpdateDonorProfileDto (areaId, phone, isAvailable).
   - If dto.areaId is provided, verifies it exists via AreasService.findByIdOrThrow(dto.areaId).
   - If dto.phone is provided, normalizes it and checks that no other User has that phone (409 An account with this phone number is already registered.).
   - Updates User.phone and DonorProfile (areaId, isAvailable) inside a transaction and returns the updated DonorDashboardProfile (FR-3, R-24).
3. **deactivateMyAccount(userId) (PATCH /api/donors/me/deactivate, @Roles('DONOR'))**:
   - Inside prisma.$transaction: sets user.isActive = false, donorProfile.isAvailable = false, and updates any active 'PLEDGED' pledge of this donor to 'WITHDRAWN' (FR-3, R-24).
   - Returns { message: 'Your account has been deactivated.' }.
4. **getMyEligibleRequests(userId) (GET /api/donors/me/eligible-requests, @Roles('DONOR'))**:
   - Loads the signed-in donor profile and queries all OPEN requests in the donor's area.

    - Filters requests using MatchingService.checkDonorEligibilityForRequest(donor, request, ClockService.now()).eligible === true (Flow C, FR-10).
    - Orders matching requests by urgency (CRITICAL → HIGH → NORMAL) and then createdAt ASC.
    - Computes canPledge = !donorHasActivePledge && activePledgesCount < unitsStillNeeded.
5. **findAllDonors() (GET /api/donors, @Roles('COORDINATOR'))**:
    - Returns all donors ordered by fullName ASC with calculated nextAllowedDonationDate and eligibilityStatus (Section 5 Role Table).

Part B — Pledges Service & Controller (api/src/pledges/)

1. **createPledge(requestId, userId) (POST /api/requests/:id/pledges, @Roles('DONOR'))**:
    - Checks in this **exact order** (R-15, R-16, R-17):
        1. Request exists (404 Blood request not found.).
        2. Request status === RequestStatus.OPEN (409 Pledges can only be made for OPEN requests. — R-9, R-15).
        3. Donor does not already hold an active PLEDGED pledge across **any** request (409 You already have an active pledge. You can hold at most one active pledge at a time. — R-16).
        4. Donor is eligible for this request via MatchingService.checkDonorEligibilityForRequest(donor, request, ClockService.now()) (R-6, R-15). If ineligible, throws BadRequestException(check.forDonor) (which includes nextAllowedDonationDate when in rest period: "You cannot pledge while in rest period. Your next allowed donation date is YYYY-MM-DD.").
        5. Count of active PLEDGED pledges on this request is < (request.unitsRequired - request.unitsCollected) (409 This request already has enough active pledges for all remaining units needed. — R-17).
    - Creates the Pledge with status: PledgeStatus.PLEDGED and returns DonorActivePledge (201 Created).
2. **withdrawPledge(pledgeId, userId) (PATCH /api/pledges/:id/withdraw, @Roles('DONOR'))**:
    - Loads the pledge (404 Pledge not found.).
    - Verifies pledge.donor.userId === userId (403 You can only withdraw your own pledge. — R-24).
    - Verifies pledge.status === PledgeStatus.PLEDGED (409 Only active PLEDGED pledges can be withdrawn.).
    - Updates status to PledgeStatus.WITHDRAWN and returns { id: pledge.id, status: 'WITHDRAWN' } (Flow C, FR-11).

Part C — Endpoint Testing (api/requests/api.http)

Write a comprehensive VS Code REST Client file (api/requests/api.http) covering all 25 items in

docs/API.md, section 19.

- Run api.http after every backend PR merge and report any status or message mismatch in the team chat.
- Verify that every error response has { "message": "..." } and matches docs/API.md verbatim.

Test Table (Laith)

Case	Expected Result
Eligible donor pledges to an OPEN request	201 Created, status: 'PLEDGED'
Same donor tries to pledge to a second OPEN request	409 You already have an active pledge. You can hold at most one active pledge at a time.
Donor in rest period tries to pledge	400 You cannot pledge while in rest period. Your next allowed donation date is YYYY-MM-DD.
Donor marked unavailable tries to pledge	400 You cannot pledge while your availability is set to unavailable.
Donor with A+ blood tries to pledge to A- request	400 Your blood type (A+) is not compatible with this request (A-).
Donor in Damascus tries to pledge to Homs request	400 You can only pledge to requests in your registered area (Damascus).
Request needs 1 more unit and already has 1 PLEDGED pledge; second donor tries to pledge	409 This request already has enough active pledges for all remaining units needed.
Donor withdraws their PLEDGED pledge, then pledges again	Both succeed (200 OK then 201 Created)
Donor tries to withdraw another donor's pledge	403 You can only withdraw your own pledge.

Laith must NOT

- duplicate blood compatibility or 56-day rest period calculations (always use Eyad's MatchingService)
- allow a donor to read or modify another donor's profile or pledge (always scope by @CurrentUser())
- allow coordinators to edit a donor's personal profile (Section 5 Role Table)
- write frontend code or edit files outside api/src/donors/, api/src/pledges/, and api/requests/api.http

Definition of Done (Laith)

- GET /api/donors/me, PATCH /api/donors/me, PATCH /api/donors/me/deactivate, GET /api/donors/me/eligible-requests, and GET /api/donors work with exact shapes and Swagger annotations.
- POST /api/requests/:id/pledges and PATCH /api/pledges/:id/withdraw enforce R-15, R-16, R-17, and R-24 in exact order with exact messages.
- api/requests/api.http covers all 25 test cases from docs/API.md, section 19, and passes against the integrated backend.

9. Task 4 — Molham Al Kasir

Frontend Auth, Public SOS Board & Shared UI (client/src/)

Main responsibility

Build the shared UI components and formatting utilities used by every frontend page, the AuthContext managing JWT sign-in/sign-up/session persistence, the **Public SOS Board (/)**, the **Sign In (/sign-in)** and **Donor Sign Up (/sign-up)** pages, and the **Not Found (*)** page.

Files

See Molham's file list in section 4.

Part A — Shared UI Components & Utilities (First Small PR)

Taymaa and Rami depend on these components and utilities, so merge them within the first **1.5 to 2 hours of Phase 1**:

Item	Behavior
Loading	Accessible spinner and optional message
ErrorMessage	Crimson alert box showing message and an optional **Try again** button (onRetry)
EmptyState	Clean empty card with title, optional description, and optional action node
Button	Variants primary, secondary, danger, outline; sizes sm, md, lg; isLoading disables button and shows loading text
StatusBadge	Renders semantic badge + text label for RequestStatus, PledgeStatus, or EligibilityStatus
UrgencyBadge	Renders CRITICAL (--color-critical), HIGH (--color-warning), or NORMAL (--color-info)
BloodTypeBadge	Renders prominent blood-type badge (O-, A-, etc.) and optional Exact Match tag
formatDate(dateStr)	'2026-12-27' → '27 Dec 2026'
formatDateTime(isoStr)	'2026-11-01T09:10:00.000Z' → '01 Nov 2026, 09:10'
formatUnits(units)	1 → '1 unit', 3 → '3 units'
toDateString(date) / getTodayString()	Local Date → 'YYYY-MM-DD' (never using toISOString())

Item	Behavior
constants.ts	BLOOD_TYPES, URGENCY_LEVELS, REQUEST_STATUSES, MIN_UNITS = 1, MAX_UNITS = 10

Part B — AuthContext (client/src/context/AuthContext.tsx)

Implement the fixed AuthContext shape (PROJECT-STRUCTURE.md, section 5.2):

1. On mount, check getStoredToken(). If a token exists, call loadCurrentUser() (GET /api/auth/me). If /auth/me fails with 401/403, call clearStoredToken() and set user = null.
2. signIn(payload): calls signIn(payload) from auth.api.ts, stores accessToken via setStoredToken, updates user, and returns user (or sets authError and returns null).
3. signUp(payload): calls signUpDonor(payload) from auth.api.ts, stores accessToken, updates user, and returns user (or sets authError and returns null).
4. signOut(): clears stored token and resets user = null.
5. loadAreas(): fetches GET /api/areas and stores areas.

Part C — Public SOS Board (/ — SosBoardPage.tsx)

1. Calls loadSosRequests() from useRequests() on mount.
2. Shows the dark crimson hero section with mission statement and quick actions (**Become a Donor**, **Sign In**, or **My Dashboard** when signed in).
3. Renders the list of PublicSosRequest items in a responsive card grid (SosRequestCard).
4. Each card shows **only public fields** (FR-8, R-23): BloodTypeBadge, UrgencyBadge, area name, hospital name, unitsStillNeeded ("3 units still needed"), and formatDateTime(createdAt).
5. Includes a **Can you help? Sign in to pledge** link on each card for visitors (or a link to /donor/dashboard for signed-in donors).
6. Handles Loading, ErrorMessage (with retry), and EmptyState when sosRequests.length === 0 (Section 13: "The SOS board when there are no open requests — Empty state, not an error").

Part D — Sign In (/sign-in) & Donor Sign Up (/sign-up)

1. **SignInPage.tsx**:
    - Fields: identifier (phone number or email) and password.
    - Submit button disabled while isAuthLoading is true.
    - Displays authError (e.g., Invalid credentials. Please check your phone/email and password.) in an inline error banner next to the form (never only in console).
    - On success, navigates to /coordinator/requests if user.role === 'COORDINATOR' or /donor/dashboard if user.role === 'DONOR'.
2. **SignUpPage.tsx**:
    - Calls loadAreas() on mount to populate the Area <select>.
    - Fields: fullName, phone, email (optional), password, bloodType (BLOOD_TYPES dropdown), and areaId (areas dropdown).
    - Shows backend validation (400) and conflict (409 duplicate phone/email) messages

inline inside the form, keeping the user's typed inputs intact.
    - On success, navigates to /donor/dashboard.

Part E — Not Found (* — NotFoundPage.tsx)

Friendly 404 screen with a button returning to the Public SOS Board (/).

Test Table (Molham)

Situation	Expected Behavior
Open / with open requests	Shows CRITICAL first, then HIGH, then NORMAL, with blood type, area, hospital, and unitsStillNeeded (no patient references or donor names)
Open / when zero open requests exist	Shows EmptyState, not an error
Sign up with missing field or duplicate phone	Shows exact backend 400 or 409 message next to the form
Sign in with wrong password	Shows Invalid credentials. Please check your phone/email and password.
Refresh browser while signed in	Token in localStorage restores user via /api/auth/me

Molham must NOT

- build the Donor Dashboard, Donor Profile, or Coordinator screens
- calculate blood compatibility or rest periods
- write backend code
- use hardcoded hex/RGB colors outside theme.css or unprefixed CSS classes
- call fetch or services/ directly inside pages or components

Definition of Done (Molham)

- Shared UI components and utilities are merged early and match the fixed props in PROJECT-STRUCTURE.md, section 7.
- AuthContext matches the fixed shape and persists JWT sessions across page refreshes.
- Public SOS Board (/) shows all required public fields, urgency ordering, and loading/empty/error states.
- Sign In (/sign-in) works for both Coordinator and Donor roles and redirects appropriately.
- Donor Sign Up (/sign-up) loads areas from the API and displays all validation/conflict messages inline.
- CSS uses sos-board-, sign-in-, sign-up-, not-found-, and ui- prefixes and theme.css variables only.

10. Task 5 — Taymaa Watfeh

Frontend Donor Experience

(client/src/pages/DonorDashboard/, DonorProfile/, DonorContext.tsx)

Main responsibility

Build the complete volunteer donor experience: DonorContext, the **Donor Dashboard (/donor/dashboard)** showing eligibility status, next allowed donation date, active pledge with withdraw action, eligible open requests with pledge action, and donation history, plus the **Donor Profile (/donor/profile)** page for editing area, phone, availability, and deactivating the account.

Files

client/src/pages/DonorDashboard/
├── DonorDashboardPage.tsx
├── DonorDashboardPage.css
├── EligibilityBanner.tsx
├── ActivePledgeCard.tsx
├── EligibleRequestsList.tsx
└── DonationHistoryTable.tsx

client/src/pages/DonorProfile/
├── DonorProfilePage.tsx
└── DonorProfilePage.css

client/src/context/DonorContext.tsx
client/src/services/donors.api.ts
client/src/services/pledges.api.ts
client/src/types/donor.types.ts

Part A — DonorContext (client/src/context/DonorContext.tsx)

Implement the fixed DonorContext shape (PROJECT-STRUCTURE.md, section 5.2):

Action	Behavior
loadMyProfile()	Calls getMyDonorProfile(), stores profile, catches errors into profileError.
loadEligibleRequests()	Calls getMyEligibleRequests(), stores eligibleRequests, catches errors into eligibleRequestsError.
updateMyProfile(payload)	Calls updateMyDonorProfile(payload), updates profile, and calls loadEligibleRequests() (since changing area or availability changes eligible requests). Returns true on success, false on error (actionError).

Action	Behavior
deactivateMyAccount()	Calls deactivateMyAccount(), returns true on success, false on error.
createPledge(requestId)	Calls createPledge(requestId) (POST /api/requests/:id/pledges). On success, immediately calls loadMyProfile() and loadEligibleRequests() so the active pledge card and request counters update without a page reload. On failure, stores the exact backend message in actionError and returns false.
withdrawPledge(pledgeId)	Calls withdrawPledge(pledgeId) (PATCH /api/pledges/:id/withdraw). On success, immediately calls loadMyProfile() and loadEligibleRequests(). On failure, stores actionError and returns false.

Part B — Donor Dashboard (/donor/dashboard — DonorDashboardPage.tsx)

1. Calls loadMyProfile() and loadEligibleRequests() on mount.
2. **EligibilityBanner.tsx**:
    - Displays the donor's blood type (BloodTypeBadge), area, StatusBadge (profile.eligibilityStatus), lastDonationDate (or "No previous donations"), and nextAllowedDonationDate:
        - When eligibilityStatus === 'ELIGIBLE': green banner — "You are eligible to donate now."
        - When eligibilityStatus === 'REST_PERIOD': amber warning banner — "You are in your 56-day rest period until [formatDate(nextAllowedDonationDate)]."
        - When eligibilityStatus === 'UNAVAILABLE': muted banner — "Your status is set to Unavailable. Update your profile when you are ready to donate." with a link to /donor/profile.
3. **ActivePledgeCard.tsx**:
    - If profile.activePledge is present, displays the pledged request's blood type, urgency badge, hospital, area, unitsStillNeeded, pledge timestamp, and a **Withdraw Pledge** button (variant="danger", isLoading={isActionLoading}).
    - If profile.activePledge === null, shows a compact note that the donor has no active pledge.
4. **EligibleRequestsList.tsx ("Requests I Can Help With")**:
    - Displays eligibleRequests returned by GET /api/donors/me/eligible-requests (FR-10).
    - Each card shows BloodTypeBadge, UrgencyBadge, hospital, area, unitsRequired, unitsCollected, unitsStillNeeded, and a **Pledge to Donate** button.
    - Displays any actionError (such as already holding an active pledge or rest-period rejection) clearly above the list.
    - Shows EmptyState when there are no matching open requests (explaining why if the donor is in rest period or unavailable).

5. **DonationHistoryTable.tsx**:
    - Lists profile.donationHistory (formatDate(donationDate), hospital, area, blood type, 1 unit) or an EmptyState ("No recorded donations yet").

Part C — Donor Profile (/donor/profile — DonorProfilePage.tsx)

1. Calls loadMyProfile() and loadAreas() (from useAuth()) on mount.
2. Shows read-only donor summary: full name, blood type badge, email, last donation date, next allowed donation date.
3. **Edit Profile Form**:
    - Area selector (areaId), phone number input (phone), and Availability toggle (isAvailable: **Available to donate** vs. **Currently unavailable**) (FR-3).
    - Shows actionError inline if the backend rejects the phone number (400 or 409), and shows a clear success banner when saved.
4. **Deactivate Account (Danger Zone)**:
    - Button **Deactivate My Account** asks for confirmation first ("Deactivating your account will withdraw any active pledge and remove you from all matching lists.").
    - When confirmed, calls deactivateMyAccount() and signOut(), then navigates to /.

Test Table (Taymaa)

Situation	Expected Behavior
Sara (A-, Homs, eligible) opens /donor/dashboard	Green ELIGIBLE banner, sees the Homs A- CRITICAL request under "Requests I can help with"
Sara clicks **Pledge to Donate**	Active Pledge card appears immediately with the request; counter updates without reloading the page
Sara clicks **Withdraw Pledge**	Active Pledge card clears immediately; **Pledge to Donate** button is re-enabled
After coordinator records Sara's donation	Dashboard shows amber REST_PERIOD banner with nextAllowedDonationDate (56 days later), donation appears in Donation History, and "Requests I can help with" is empty
Donor toggles availability to Unavailable on /donor/profile	Profile saves, dashboard banner updates to UNAVAILABLE

Taymaa must NOT

- calculate nextAllowedDonationDate or filter compatible requests on the frontend (always use the backend fields from GET /api/donors/me and GET /api/donors/me/eligible-requests)
- build Coordinator screens or Auth screens
- write backend code
- call fetch or services/ directly inside components
- use hardcoded colors or unprefixed CSS classes

Definition of Done (Taymaa)

- DonorContext matches the fixed shape and refreshes profile and eligible requests automatically after pledging, withdrawing, or updating profile.
- Donor Dashboard shows eligibility status (ELIGIBLE / REST_PERIOD until date / UNAVAILABLE), nextAllowedDonationDate, active pledge with withdraw action, eligible requests with pledge action, and donation history.
- Donor Profile allows updating areaId, phone, and isAvailable, and deactivating the account with confirmation.
- Loading, empty, and error states with retry exist on both pages.
- CSS uses donor-dashboard- and donor-profile- prefixes and theme.css variables only.

11. Task 6 — Rami

Frontend Coordinator Experience (client/src/pages/Requests/, RequestDetails/, EligibleDonors/, RequestsContext.tsx)

Main responsibility

Build the complete blood-bank coordinator experience: RequestsContext, the **Coordinator Requests Page (/coordinator/requests)** with multi-filter bar and Create SOS Request modal, the **Request Details Page (/coordinator/requests/:requestId)** with unit counters, pledges list, donations list, Update/Cancel/Record Donation actions, and the **Eligible Donors Page (/coordinator/requests/:requestId/eligible-donors)** showing the ranked eligible donors and the excluded-donor summary by reason.

Files

client/src/pages/Requests/
├── RequestsPage.tsx
├── RequestsPage.css
├── RequestFiltersBar.tsx
└── CreateRequestModal.tsx

client/src/pages/RequestDetails/
├── RequestDetailsPage.tsx
├── RequestDetailsPage.css
├── UpdateRequestForm.tsx
├── CancelRequestModal.tsx
└── RecordDonationModal.tsx

client/src/pages/EligibleDonors/

├── EligibleDonorsPage.tsx
├── EligibleDonorsPage.css
├── ExcludedDonorsSummary.tsx
└── EligibleDonorRow.tsx

client/src/context/RequestsContext.tsx
client/src/services/requests.api.ts
client/src/types/request.types.ts

Part A — RequestsContext (client/src/context/RequestsContext.tsx)

Implement the fixed RequestsContext shape (PROJECT-STRUCTURE.md, section 5.2):

- loadSosRequests(): calls getSosRequests() for the public SOS board.
- loadRequests(filters?): calls getRequests(filters) for the Coordinator Requests page.
- loadRequestById(requestId): calls getRequestById(requestId) for the Request Details page.
- loadEligibleDonors(requestId): calls getEligibleDonors(requestId) for the Eligible Donors page.
- loadAllDonors(): calls getAllDonors() from donors.api.ts.
- createRequest(payload): calls createRequest(payload). On success, reloads loadRequests() and returns the created request; on failure, saves mutationError and returns null.
- updateRequest(requestId, payload): calls updateRequest(requestId, payload). On success, reloads loadRequestById(requestId) so unit counters, status (FULFILLED), and released pledges update immediately without reload.
- cancelRequest(requestId, payload): calls cancelRequest(requestId, payload). On success, reloads loadRequestById(requestId).
- recordDonation(requestId, payload): calls recordDonation(requestId, payload). On success, reloads loadRequestById(requestId) (and loadEligibleDonors(requestId) if loaded) so collected units, completed/released pledges, and FULFILLED status update immediately without reload.

Part B — Coordinator Requests Page (/coordinator/requests — RequestsPage.tsx)

1. **Filter Bar (RequestFiltersBar.tsx)**:
    - Dropdowns for status (All, OPEN, FULFILLED, CANCELLED), bloodType (All, O-..AB+), areaId (All + areas from useAuth()), and urgency (All, CRITICAL, HIGH, NORMAL) (FR-7), plus a **Reset Filters** button.
    - Changing any filter calls loadRequests(filters).
2. **Create SOS Request (CreateRequestModal.tsx)**:
    - Fields: patientReference (with helper text: "Internal hospital code — never enter a patient's personal name"), bloodType, unitsRequired (number 1–10), areaId, urgency, hospital, and optional note (FR-5).

    - Displays mutationError (such as 409 An open request for this patient reference and blood type already exists. or 400 Units required must be a whole number between 1 and 10.) clearly inside the form.
    - On success, closes the modal and refreshes the requests list (or opens the new request).
3. **Requests List**:
    - Shows each request's patientReference, BloodTypeBadge, UrgencyBadge, StatusBadge, area, hospital, units progress (unitsCollected / unitsRequired and unitsStillNeeded), active pledges count, and action links: **View Details** (/coordinator/requests/:id) and—when status === 'OPEN'—**Eligible Donors** (/coordinator/requests/:id/eligible-donors).
    - Includes Loading, ErrorMessage (with retry), and EmptyState.

Part C — Request Details Page (/coordinator/requests/:requestId — RequestDetailsPage.tsx)

1. Calls loadRequestById(requestId) on mount.
2. **Header & Unit Counters Card**:
    - Displays patientReference, BloodTypeBadge, UrgencyBadge, StatusBadge, hospital, area, note, cancelReason (if CANCELLED), and three prominent counter boxes (FR-13):
        - **Units Required** (unitsRequired)
        - **Units Collected** (unitsCollected)
        - **Units Still Needed** (unitsStillNeeded)
3. **Coordinator Action Bar (shown when status === 'OPEN', R-9, R-14)**:
    - **Find Eligible Donors** button → /coordinator/requests/:requestId/eligible-donors.
    - **Update Request (UpdateRequestForm.tsx)**: edit unitsRequired, urgency, note. Shows mutationError inline if unitsRequired < unitsCollected (400, R-12).
    - **Cancel Request (CancelRequestModal.tsx)**: requires cancelReason (PATCH /api/requests/:id/cancel, Flow E).
    - **Record Donation (RecordDonationModal.tsx)**: select donor (pre-filled when clicked from a pledge row, or chosen from eligible donors) and donationDate (defaults to getTodayString(), max={getTodayString()}). Shows exact backend rejection reason (mutationError) if the donor is ineligible or in rest period (R-19, R-21).
    - When status is FULFILLED or CANCELLED, displays a read-only banner explaining that completed or cancelled requests cannot be modified, pledged to, or receive donations (R-9, R-14).
4. **Pledges Section**:
    - Table/list of currentRequest.pledges showing donor name, phone, BloodTypeBadge, StatusBadge (PLEDGED, COMPLETED, WITHDRAWN, RELEASED), timestamp, and a quick **Record Donation** button on any PLEDGED row when the request is OPEN.
5. **Recorded Donations Section**:
    - Table/list of currentRequest.donations showing donor name, blood type,

formatDate(donationDate), 1 unit, and coordinator contact.

Part D — Eligible Donors Page (/coordinator/requests/:requestId/eligible-donors — EligibleDonorsPage.tsx)

1. Calls loadRequestById(requestId) and loadEligibleDonors(requestId) on mount.
2. **Request Context Header**: shows patient reference, needed blood type, area, urgency, unitsStillNeeded, and a **Back to Request Details** link.
3. **Excluded Donors Summary (ExcludedDonorsSummary.tsx — FR-9, R-8)**:
   - Renders three summary stat cards explaining why compatible donors in the same area were excluded from the list:
     - **In Rest Period (excludedSummary.restPeriod)**: "Compatible donors in area inside their 56-day cooldown"
     - **Unavailable (excludedSummary.unavailable)**: "Compatible donors in area currently marked unavailable"
     - **Inactive (excludedSummary.inactive)**: "Deactivated donor accounts in area"
4. **Ranked Eligible Donors Table (EligibleDonorRow.tsx — FR-9, R-7)**:
   - Renders eligibleDonors in the exact order returned by the backend.
   - Each row displays: #rank, donor fullName, phone, BloodTypeBadge (with an **Exact Match** badge when isExactBloodMatch === true or **Compatible** tag when false), area.name, lastDonationDate (formatDate(lastDonationDate) or "Never donated"), an **Active Pledge** tag if hasActivePledgeForThisRequest === true, and a **Record Donation** button that opens the record-donation modal for that donor.
   - Shows EmptyState if no eligible donors are currently available in that area.

Test Table (Rami)

Situation	Expected Behavior
Create duplicate OPEN request with same patientReference + bloodType	Modal stays open and displays An open request for this patient reference and blood type already exists.
Open Eligible Donors for Homs A- request	Shows A- donors before O- donors, never-donated / oldest donation first, and shows excludedSummary.restPeriod count for Rami (O-, donated 20 days ago)
Record donation for Sara on Homs A- request (1 of 3 units)	Counters immediately update to 1 collected / 2 still needed, Sara's pledge becomes COMPLETED, and Sara disappears from Eligible Donors (restPeriod count increases by 1)
Record 3rd donation on 3-unit request	Request status badge becomes FULFILLED, unitsStillNeeded becomes 0, remaining PLEDGED pledges become RELEASED, and Update/Cancel/Record Donation actions lock

Rami must NOT

- filter, check compatibility, or sort eligible donors on the frontend (always render eligibleDonorsResult as returned by the backend)
- calculate unitsStillNeeded on the frontend
- build Donor or Auth pages
- write backend code
- call fetch or services/ directly inside pages or components
- use hardcoded colors or unprefixed CSS classes

Definition of Done (Rami)

- RequestsContext matches the fixed shape and keeps request details, counters, and eligible donors in sync after every mutation without page reloads.
- Coordinator Requests page supports all 4 filters and the Create SOS Request modal.
- Request Details page shows all fields, counters, pledges, donations, and Update / Cancel / Record Donation actions.
- Eligible Donors page displays the ranked donor list and the 3 excluded-donor summary cards (restPeriod, unavailable, inactive).
- All backend validation and business-rule errors (400 / 409) are displayed clearly next to the forms.
- CSS uses requests-, request-details-, and eligible-donors- prefixes and theme.css variables only.

12. Shared Task — Final Integration Test

When all six tasks are merged into main, the entire team runs the end-to-end verification on a clean checkout against PostgreSQL (covering Section 6, Section 13, and Section 19 of the specification):

1. Visitor opens Public SOS Board (/) → sees OPEN requests ordered by urgency, with zero personal data.
2. Coordinator signs in (/sign-in) → publishes a new SOS request: A- blood, 3 units, area Homs, urgency CRITICAL (Flow A).
3. Public SOS Board (/) immediately shows the new Homs A- CRITICAL request.
4. Coordinator opens Eligible Donors (/coordinator/requests/:id/eligible-donors) (Flow B) → sees only active, available, non-resting A- and O- donors in Homs in the required rank order, and sees excludedSummary explaining that Rami (O-, donated 20 days ago) is excluded due to restPeriod.
5. Sara (A-, Homs) signs in (/sign-in) → opens Donor Dashboard (/donor/dashboard) → sees ELIGIBLE status and the CRITICAL Homs A- request under "Requests I can help with".
6. Sara clicks Pledge to Donate (Flow C) → her Active Pledge card appears; she tries to pledge to a second request and is rejected (409).
7. Coordinator opens Request Details (/coordinator/requests/:id) → sees Sara's PLEDGED pledge → clicks Record Donation for Sara (Flow D).
8. Request now shows 1 of 3 units collected (2 still needed), Sara's pledge is COMPLETED,

and Sara's dashboard shows she is in REST_PERIOD until 56 days later (and she no longer appears in Eligible Donors).
9. Coordinator records the remaining 2 units from other eligible donors →
on the 3rd unit, the request automatically becomes FULFILLED, disappears from the Public SOS Board,
and any remaining PLEDGED pledges become RELEASED.
10. Restart the NestJS backend server → verify all data is still persisted in PostgreSQL.

Also verify every item in the **Section 19 Acceptance Checklist**:

- [ ] A donor can sign up, sign in, and see their eligibility status and next allowed donation date.
- [ ] A donor calling a coordinator-only endpoint receives a 403 Forbidden error.
- [ ] A new request appears on the public SOS board, which shows no personal data.
- [ ] The eligible-donor list for an A- request contains only A- and O- donors from the same area, outside rest period, available and active, in the required order.
- [ ] The excluded-donor summary counts donors excluded for rest period, unavailability, and inactivity.
- [ ] A donor in rest period cannot pledge, and the message gives the next allowed date.
- [ ] A donor with an active pledge cannot pledge again.
- [ ] Recording a donation increases units collected by 1, completes the pledge, and starts the donor's configured rest period (56 days in this project).
- [ ] Recording the last needed unit marks the request FULFILLED and releases remaining pledges.
- [ ] If any step of recording a donation fails, nothing is saved (demonstrated or explained with the prisma.$transaction code).
- [ ] FULFILLED and CANCELLED requests reject updates, pledges, and donations (409 Conflict).
- [ ] Data is still there after restarting the backend.
- [ ] Swagger (/api/docs) documents every endpoint and its authentication.
- [ ] Meaningful log lines appear in the backend console when a request is created/cancelled/fulfilled and when a donation is recorded or rejected.

13. Shared Task — README

Owner: **Eyad Ibrahim**. Reviewed by all team members.
A clean checkout must run by following README.md step by step. It must cover:

- Project overview and disclaimer (coordination platform, not a medical screening system)
- Technology stack (React + TypeScript, NestJS, PostgreSQL + Prisma, JWT, Swagger)
- Folder structure (api/, client/, docs/)
- Prerequisites (Node.js, PostgreSQL) and environment variables (api/.env and client/.env)
- How to run Prisma migrations and seed data (npx prisma migrate dev, npx prisma db seed) with the seeded Coordinator and Donor demo credentials
- How to start the backend (npm run start:dev in api/, Swagger at http://localhost:3000/api/docs) and frontend (npm run dev in client/)
- Key business rules (red-cell compatibility table, 56-day rest period, eligibility & ranking, pledge limits, atomic donation transaction)
- Team table and file ownership summary

14. Shared Task — Final Presentation (English)

Per Section 18 of the specification, the team presents **in English**, **every member speaks**, and **any team member may be asked to explain any part of the project**.
The presentation must cover all 10 required points from Section 18:

#	Specification Requirement (Section 18)	Speaker	What to Show / Explain
1	The problem and the product idea (+ medical disclaimer)	**Eyad Ibrahim**	Why social media/phone calls fail during hospital blood emergencies and how Blood Donation Network matches eligible donors fast
2	Architecture & code organization (Controller → Service → Persistence) + Technical decision	**Eyad Ibrahim**	NestJS modular design, central MatchingService, why unitsCollected is stored on BloodRequest and updated inside prisma.$transaction
3	Frontend behavior (Public SOS Board, Auth, loading/empty/error states)	**Molham Al Kasir**	Live tour of / SOS Board (showing zero personal data, R-23), Sign Up validation feedback, and loading/empty/error states
4	Main user flow Part 1: Publish request → Find eligible donors + Compatibility & Ranking rules	**Rami**	Coordinator publishes Homs A- request, opens Eligible Donors (A- and O- ranked order) and explains the excluded-donor summary (restPeriod, unavailable, inactive)
5	Main user flow Part 2: Donor Dashboard → Pledge → Rest Period rule	**Taymaa Watfeh**	Sara signs in, sees ELIGIBLE banner, pledges to the Homs A- request, and shows what happens after donation

#	Specification Requirement (Section 18)	Speaker	What to Show / Explain
			(REST_PERIOD until lastDonationDate + 56 days)
6	Backend/API behavior in Swagger + Pledge & Role validation rules	**Laith Haj Hosin**	Demonstrates /api/docs (Swagger), shows a 403 Forbidden role check and a 400/409 pledge rejection (rest period with exact date or duplicate active pledge)
7	Main user flow Part 3: Record donation in a transaction → Fulfilled + Persisted data in PostgreSQL + Logger + Future improvement	**Adham Albasha**	Records the donation on screen, shows prisma.$transaction code (R-22), AppLogger console lines, persisted rows in Prisma Studio / PostgreSQL after restart, and one limitation / next improvement (e.g., Stretch shortage forecast or request expiry)

15. Git Workflow

Every member works on a dedicated feature branch for each task.

Before you start (every time)

git switch main
git pull origin main
git switch -c be/requests-crud
git branch --show-current

The last command must **never** print main.

Branch names

Format: be/<module>-<description> or fe/<page>-<description>.

Member	Branches
Eyad	be/common-infrastructure, be/auth-and-areas,

Member	Branches
	be/matching-engine, fe/layout-and-protected-route, docs/readme-and-polish
Adham	be/requests-dtos-and-sos, be/requests-crud-and-cancel, be/requests-eligible-donors, be/donations-transaction
Laith	be/donors-profile-and-list, be/donors-eligible-requests, be/pledges-module, be/api-http-tests
Molham	fe/shared-ui-and-utils, fe/auth-context-and-pages, fe/sos-board-page, fe/not-found-page
Taymaa	fe/donor-context, fe/donor-dashboard-page, fe/donor-profile-page
Rami	fe/requests-context, fe/coordinator-requests-page, fe/request-details-page, fe/eligible-donors-page

Commit messages

Format: type(scope): message (feat, fix, refactor, style, docs, chore)
feat(matching): implement red-cell compatibility and 56-day rest period check
feat(donations): record donation atomically inside prisma transaction
feat(pledges): reject pledge when donor already holds an active pledge
feat(donor-dashboard): display eligibility banner and active pledge card
fix(requests): prevent reducing unitsRequired below unitsCollected

Pull Request template (.github/pull_request_template.md)

What does this PR do?

Area
- [ ] Frontend (`client/`)
- [ ] Backend (`api/`)
- [ ] Docs / Config

How did you test it?

Screenshots (frontend or Swagger/Logger output)

Checklist
- [ ] I am not on `main`
- [ ] I changed only files I own (`docs/TASKS.md`, section 4)

- [ ] Names, routes, and DTO orders follow `docs/PROJECT-STRUCTURE.md` and `docs/API.md`
- [ ] Controllers have zero business rules and zero Prisma queries
- [ ] No hardcoded colors; CSS classes use my BEM owner prefix
- [ ] Loading, empty, and error states are handled
- [ ] Both `api` and `client` build cleanly

CODEOWNERS (.github/CODEOWNERS)

* @eyad-ibrahim

Branch protection on main (Eyad, once after Phase 0)

In GitHub **Settings** → **Branches / Rulesets** for main:

- Require a pull request before merging
- Require 1 approval (Eyad approves team PRs; Adham approves Eyad's PRs)
- Dismiss stale approvals when new commits are pushed
- Block force pushes and branch deletion
- Do not allow bypassing branch protection

16. Git Rules

- **Never push directly to main.**
- Run git branch --show-current before every commit.
- One task = one branch. Keep PRs small and focused (1–2 PRs per day).
- Pull and merge origin/main into your branch before opening a PR (git fetch origin && git merge origin/main).
- Edit only files you own (section 4).
- Never commit node_modules, dist, .env, or database credentials.
- Never run npm install <package> without asking Eyad first.
- Code freeze takes effect **2 hours before the presentation**: only critical bug fixes approved by Eyad are allowed.

17. Order of Work

The project is executed in phases over **3 days**:

Phase 0 — Foundation (Eyad) — First 2 to 2.5 hours of Day 1

- Repo setup, prisma/schema.prisma, initial migration, prisma/seed.ts, NestJS + Swagger + common/ + stubs in api/, React + Router + Context skeletons + http.ts + theme.css in client/, branch protection on main.

Phase 1 — Core Infrastructure & Parallel Feature Work — Rest of Day 1

First wave of small PRs (merge within 2–3 hours after Phase 0):

1. **Eyad**: be/common-infrastructure (validators.ts, normalize.util.ts, validation.pipe.ts, all-exceptions.filter.ts, app-logger.service.ts), be/auth-and-areas, and be/matching-engine (MatchingService)
2. **Molham**: fe/shared-ui-and-utils (Loading, ErrorMessage, EmptyState, Button, StatusBadge, UrgencyBadge, BloodTypeBadge, format.ts, date.ts, constants.ts)

Second wave of PRs (Day 1 afternoon/evening):

1. **Adham**: be/requests-dtos-and-sos, be/requests-crud-and-cancel, be/requests-eligible-donors
2. **Laith**: be/donors-profile-and-list, be/donors-eligible-requests, be/pledges-module
3. **Molham**: fe/auth-context-and-pages (AuthContext, SignInPage, SignUpPage), fe/sos-board-page
4. **Taymaa**: fe/donor-context, fe/donor-dashboard-page
5. **Rami**: fe/requests-context, fe/coordinator-requests-page

Phase 0 Foundation (Schema, Seed, Stubs, Skeletons)
↓
Wave 1: Eyad (Common + Auth + MatchingService) + Molham (Shared UI & Utils)
↓
Wave 2: Adham (Requests & Donations) + Laith (Donors & Pledges) + Molham (Auth & SOS Pages)
↓
Wave 3: Taymaa (Donor Dashboard & Profile) + Rami (Coordinator Requests, Details & Eligible Donors) + Laith (api.http)

Phase 2 — Full Integration & Transaction Verification — Morning of Day 2

- **Adham** merges be/donations-transaction (POST /api/requests/:id/donations with prisma.$transaction).
- **Taymaa** merges fe/donor-profile-page and connects all dashboard actions to real endpoints.
- **Rami** merges fe/request-details-page and fe/eligible-donors-page.
- **Laith** runs the complete 25-request suite in api/requests/api.http against the live backend and posts results.
- **Eyad** runs the Section 12 end-to-end integration test across all screens.

Phase 3 — Edge-Case Hardening & UI Polish — Afternoon of Day 2

- Verify every validation & error scenario in Section 13 and every checkbox in Section 19.
- Test backend restart persistence, Swagger documentation completeness at /api/docs, and AppLogger output.
- Verify responsive layouts on mobile and desktop viewports and stop-the-backend error

handling.

Phase 4 — Documentation & Presentation Rehearsal — Day 3 Morning

- **Eyad** finalizes README.md.
- Code freeze 2 hours before the presentation.
- Run two timed English presentation rehearsals with all 6 members speaking and demonstrating the live flow on clean seeded data.

18. Final Definition of Done

The project is complete only when all of the following are verified:

Frontend

- React + TypeScript + React Router + 3 React Contexts (AuthContext, DonorContext, RequestsContext)
- All 8 required screens exist and are reachable through navigation: Public SOS Board (/), Sign In (/sign-in), Sign Up (/sign-up), Donor Dashboard (/donor/dashboard), Donor Profile (/donor/profile), Coordinator Requests (/coordinator/requests), Request Details (/coordinator/requests/:requestId), Eligible Donors (/coordinator/requests/:requestId/eligible-donors), plus Not Found (*)
- Every screen shows loading, empty, and error states with retry
- Forms show backend validation and business-rule rejection messages (such as exact rest-period end date) next to the form
- Lists and unit counters update immediately after an action without a manual page reload
- Protected screens show a clear sign-in prompt when not signed in

Backend & Database

- NestJS + TypeScript following Controller → Service → Persistence (Prisma)
- PostgreSQL database with Prisma schema, migrations, and seed script (areas, >= 2 coordinators, >= 20 donors covering all 8 blood types including resting/unavailable/inactive donors, and requests in OPEN, FULFILLED, and CANCELLED states)
- All data survives a backend restart
- DTO validation on every body, param, and query with whitelist: true and forbidNonWhitelisted: true
- JWT authentication with JwtAuthGuard and RolesGuard enforcing COORDINATOR and DONOR roles
- Swagger / OpenAPI at /api/docs documenting every endpoint, parameter, body, and auth requirement
- Custom AppLogger logging request created, cancelled, fulfilled, donation recorded, and rejected donation attempts with reason (never logging passwords or tokens)

Business Rules

- Red-cell blood compatibility (R-1, R-2) enforced on the backend
- Single configured rest period (REST_PERIOD_DAYS = 56, R-3..R-5) enforced on the backend
- Eligible-donor matching (R-6), ranking (R-7), and excluded-donor summary (R-8) working accurately
- Request lifecycle (R-9..R-14) and pledge rules (R-15..R-18) enforced with 400 and 409 errors
- Donation recording (R-19..R-22) executed inside a single prisma.$transaction, automatically fulfilling the request and releasing remaining pledges when unitsCollected === unitsRequired
- Privacy rules (R-23, R-24) enforced on all public and donor endpoints
- Every team member contributed on their own branches and is ready to present in English